Projeto 1 — Sistema de Apoio à Decisão para Rotas Logísticas

Caminhos mínimos: Dijkstra, Bellman-Ford e Floyd-Warshall

Valor 10,0 = 1,0 Média Final

1. Contexto

A empresa ConnectLog opera uma rede de centros de distribuição interligados por rotas de transporte. O setor de Pesquisa Operacional precisa apoiar decisões sobre quais rotas utilizar para reduzir custos e como analisar a rede quando existem diferentes condições operacionais.

Cada centro ou ponto logístico é representado por um vértice. Cada rota é representada por uma aresta dirigida e ponderada. O peso representa o custo operacional líquido de transportar uma unidade de carga pela rota. Em alguns cenários, incentivos, créditos ou subsídios podem produzir custos negativos.

2. Objetivo do projeto

Desenvolver um programa que aplique algoritmos de caminhos mínimos a diferentes situações da rede logística e, ao final, produza uma recomendação técnica sobre qual algoritmo é adequado a cada tipo de problema.

- Implementar Dijkstra, Bellman-Ford e Floyd-Warshall.
- Reconstruir caminhos mínimos, e não apenas calcular seus custos.
- Identificar situações com pesos negativos e ciclos negativos.
- Comparar os algoritmos e interpretar os resultados no contexto de Pesquisa Operacional.

> Importante: o foco deste projeto é a modelagem do problema, os algoritmos em grafos e a interpretação dos resultados. Não é necessário desenvolver interface gráfica, banco de dados, API ou leitura de arquivos.

3. Cenário 1 — Operação normal

Em condições normais, todas as rotas possuem custos não negativos. Considere o grafo dirigido G1 = (V, E), com:

V = {CD, A, B, C, D, E, F, Cliente}

Origem	Destino	Custo
CD	A	4
CD	B	7
A	B	2
A	C	5
B	C	1
B	D	6
C	D	2
C	E	7
D	E	3
D	F	5
E	F	2
E	Cliente	6
F	Cliente	3

3.1 Tarefas do cenário 1

- Implemente Dijkstra e determine o menor custo de CD até Cliente.
- Apresente também a sequência de vértices que forma o caminho mínimo encontrado.
- Implemente e Execute Bellman-Ford com a mesma origem e compare o resultado para CD → Cliente.
- Implemente e Execute Floyd-Warshall e verifique, na matriz de distâncias, o valor correspondente a CD → Cliente.
- Explique por que os três algoritmos podem ser aplicados a este cenário.
- Caso existam diferentes caminhos com o mesmo custo mínimo, o programa pode apresentar qualquer um deles, desde que o custo esteja correto.

4. Cenário 2 — Incentivos e créditos logísticos

A ConnectLog passou a receber incentivos financeiros em algumas rotas. Assim, determinadas operações podem apresentar custo líquido negativo. Considere o grafo G2 = (V, E):

V = {CD, A, B, C, D, E, F, Cliente}

Origem	Destino	Custo
CD	A	2
CD	B	5
A	C	2
B	A	-4
B	D	3
C	D	2
C	E	5
D	E	2
D	Cliente	6
E	F	2
F	Cliente	1

4.1 Tarefas do cenário 2

- Implemente Bellman-Ford e determine o menor custo de CD até Cliente.
- Reconstrua e apresente o caminho correspondente.
- Implemente e Execute Floyd-Warshall e consulte também o caminho/custo de CD até Cliente. Não é necessário imprimir a matriz completa deste cenário no relatório.
- Antes de executar Dijkstra, o programa deve verificar se existem arestas de peso negativo. Caso existam, deve informar que Dijkstra não é aplicável ao cenário, em vez de fornecer um resultado potencialmente incorreto.
- Explique por que um custo negativo pode fazer sentido neste modelo mesmo que uma distância física negativa não faça sentido.

5. Caso de teste — ciclo negativo

Além dos dois cenários principais, utilize o grafo G3 abaixo apenas para testar a detecção de ciclo negativo:

```
V = {A, B, C, D}
```

Origem	Destino	Custo
A	B	3
B	C	2
C	A	-6
C	D	4

- Bellman-Ford deve detectar a existência de um ciclo negativo alcançável a partir de A.
- Floyd-Warshall deve permitir a identificação de ciclo negativo por meio da diagonal da matriz de distâncias.
- Explique por que, na presença de um ciclo negativo alcançável, não existe um menor caminho bem definido para alguns pares de vértices.

6. Requisitos de implementação

O programa deve possuir, no mínimo, implementações equivalentes às seguintes operações:

```
dijkstra(grafo, origem)
bellmanFord(grafo, origem)
floydWarshall(grafo)
reconstructPath(...)
hasNegativeEdge(...)
```

A assinatura exata dos métodos pode ser adaptada à estrutura escolhida pela dupla. O comportamento exigido, entretanto, deve ser mantido.

6.1 Saídas mínimas

- Custo mínimo calculado.
- Caminho mínimo reconstruído, quando ele estiver bem definido.
- Mensagem clara quando Dijkstra não puder ser aplicado por existir peso negativo.
- Mensagem clara quando for detectado ciclo negativo.
- Matriz de menores custos de Floyd-Warshall para o Cenário 1.

7. Código-base

Para reduzir o tempo gasto com infraestrutura, a dupla pode utilizar o código-base abaixo. Ele não contém os algoritmos avaliados no projeto.

```
import java.util.ArrayList;
import java.util.List;

class Edge {
	int from;
	int to;
	int weight;

	Edge(int from, int to, int weight) {
		this.from = from;
		this.to = to;
		this.weight = weight;
	}
}

class Graph {
	private final int vertices;
	private final List<Edge> edges = new ArrayList<>();

	Graph(int vertices) {
		this.vertices = vertices;
	}

	void addEdge(int from, int to, int weight) {
		edges.add(new Edge(from, to, weight));
	}

	int getVertices() {
		return vertices;
	}

	List<Edge> getEdges() {
		return edges;
	}
}
```

É permitido acrescentar estruturas auxiliares, como lista de adjacências, vetor de predecessores ou matriz de distâncias. Não é permitido substituir as implementações solicitadas por bibliotecas prontas de caminhos mínimos.

8. Formato esperado das saídas

A apresentação pode ser realizada no terminal. O formato abaixo é apenas uma referência; os valores devem ser calculados pelo programa.

```
CENARIO 1 – DIJKSTRA
Origem: CD
Destino: Cliente
Custo minimo: <valor calculado>
Caminho: <sequencia de vertices>

CENARIO 3 – BELLMAN-FORD
Ciclo negativo detectado. Nao existe menor caminho bem definido.
```

9. Análise de Pesquisa Operacional

Além da implementação, a dupla deverá responder no relatório às questões abaixo. Respostas devem ser justificadas com base nos algoritmos e nos resultados obtidos.

- **Situação A —** A empresa precisa calcular diariamente rotas partindo apenas do centro de distribuição principal e todos os custos são não negativos. Qual algoritmo você recomendaria? Por quê?
- **Situação B —** Algumas rotas possuem créditos que podem resultar em pesos negativos, mas não há ciclo negativo. Qual algoritmo é adequado para uma única origem?
- **Situação C —** A administração deseja conhecer os menores custos entre todos os pares de unidades. Qual algoritmo é mais apropriado?
- **Situação D —** O que um ciclo negativo representa para o problema de menor caminho?
- **Comparação —** No Cenário 1, os três algoritmos devem produzir resultados compatíveis para CD → Cliente. Compare-os e explique por que isso ocorre.
- **Decisão —** Em quais circunstâncias você evitaria Floyd-Warshall mesmo sabendo que ele resolve o problema de todos os pares?

10. Entregáveis

- Código-fonte completo do projeto em Java.
- Um arquivo README com instruções simples de compilação e execução.
- Relatório em PDF, com aproximadamente 4 a 6 páginas, contendo resultados, comparações e respostas da Seção 9.
- Evidências de execução dos três cenários, por exemplo por meio de capturas do terminal ou trechos de saída no relatório.

11. Organização sugerida do relatório

Seção	Conteúdo
1. Problema e modelagem	Descrição resumida da rede e da representação utilizada.
2. Dijkstra	Resultado do Cenário 1 e caminho encontrado.
3. Bellman-Ford	Resultados dos Cenários 1 e 2 e detecção do ciclo negativo.
4. Floyd-Warshall	Matriz do Cenário 1 e consultas realizadas.
5. Comparação	Resultados comuns e diferenças de aplicabilidade.
6. Decisão em PO	Respostas e recomendações da Seção 9.

12. Critérios de avaliação

Critério	Pontos
Representação correta dos grafos	0,5
Implementação de Dijkstra	2,0
Implementação de Bellman-Ford	2,0
Implementação de Floyd-Warshall	2,0
Reconstrução de caminhos e apresentação dos resultados	1,0
Tratamento de pesos negativos e ciclos negativos	0,5
Análise comparativa e decisões de Pesquisa Operacional	1,5
Organização e clareza do código	0,5
Total	10,0

13. Regras finais

- O trabalho deve ser realizado em dupla.
- Os grafos fornecidos no enunciado devem ser utilizados nos testes obrigatórios.
- É permitido criar testes adicionais, desde que eles não substituam os cenários obrigatórios.
- As implementações de Dijkstra, Bellman-Ford e Floyd-Warshall devem ser próprias da dupla. Não devem ser utilizadas bibliotecas que executem diretamente esses algoritmos.
- A dupla deve ser capaz de explicar o funcionamento do próprio código e justificar as escolhas feitas.
- Erros de modelagem que alterem as arestas, direções ou pesos dos grafos fornecidos serão considerados na avaliação.

O objetivo não é criar um sistema de software completo. O objetivo é utilizar algoritmos de grafos como ferramentas de apoio à decisão em diferentes condições operacionais.